

Analyzing the Role of AI-Assisted Programming in Modern Software Development

Introduction

Artificial Intelligence is increasingly integrated into modern software development through tools such as AI code generators, debugging assistants, and intelligent programming support systems. These technologies are transforming how developers design, implement, test, and maintain software applications.

This assignment is designed to help students understand the impact of AI-assisted programming in software engineering while evaluating both its technical and ethical implications. Students will explore AI-assisted development techniques, implement a programming solution using both traditional and AI-assisted approaches, and critically analyze the results.

Assignment Tasks

Task 1: Conceptual Understanding

1.1 What is AI-Assisted Programming?

AI-assisted programming refers to the use of artificial intelligence tools, typically built on large language models (LLMs), to support software developers throughout the software development lifecycle. Rather than replacing the programmer, these tools act as an intelligent collaborator that can generate code from natural-language descriptions, suggest completions, detect bugs, explain unfamiliar code, and propose refactoring options. Popular examples include GitHub Copilot, ChatGPT, Amazon CodeWhisperer, and Claude, all of which are trained on large corpora of source code and documentation and can therefore recognise common programming patterns and produce contextually relevant suggestions.

The core idea behind AI-assisted programming is to reduce repetitive manual effort, shorten development time, and lower the barrier to entry for tasks such as syntax lookup, boilerplate generation, and documentation writing, while leaving design decisions, architecture, and critical judgement to the human developer

1.2 Types of AI Coding Tools

- Code Generation Tools — generate new code from natural-language prompts or partial code context (e.g., GitHub Copilot, ChatGPT, Claude, Amazon Q Developer).
- Debugging Assistants — analyse stack traces, error messages, and logic to identify the root cause of bugs and suggest fixes (e.g., DeepCode, Copilot Chat, ChatGPT debugging mode).
- Refactoring and Code Review Tools — suggest structural improvements, detect code smells, and enforce style conventions (e.g., SonarQube AI, CodeRabbit, Copilot Workspace).
- Documentation Generators — automatically produce docstrings, README files, and inline comments from existing code.
- Test Generation Tools — create unit tests and edge-case scenarios based on function signatures and code behaviour.

1.3 Advantages in Software Development

- Faster development — boilerplate and repetitive code can be generated in seconds, freeing time for design and problem-solving.
- Lower entry barrier — beginners can learn syntax and patterns faster with contextual explanations.
- Improved productivity — autocomplete and inline suggestions reduce context-switching to search engines or documentation.
- Better documentation — AI can generate consistent docstrings and comments, improving long-term maintainability.
- Rapid prototyping — ideas can be turned into working proof-of-concept code very quickly.

- Knowledge support — AI tools can explain unfamiliar libraries, error messages, or legacy code.

1.4 Risks and Limitations

- Hallucinated code — AI models can generate code that looks correct but calls non-existent functions, libraries, or APIs.
- Over-reliance and skill erosion — excessive dependency on AI suggestions can weaken a developer's own problem-solving and debugging skills over time.
- Security vulnerabilities — generated code may contain insecure patterns (e.g., SQL injection risk, hard-coded credentials, missing input validation) if not carefully reviewed.
- Licensing and intellectual-property concerns — AI-generated code may resemble copyrighted training data, raising questions about code provenance.
- Lack of contextual understanding — AI tools do not fully understand a project's broader business logic, architecture, or long-term goals.
- False confidence — plausible-looking but subtly incorrect code can pass casual review, especially for developers unfamiliar with the domain.

Task 2: Practical Implementation –Library Management System

2.2 Version A – Traditional Programming Approach

```
import java.util.ArrayList;
import java.util.Scanner;

public class LibraryManagementSystem {

    static class Book {
        String title;
        String author;
        int copies;

        Book(String title, String author, int copies) {
            this.title = title;
            this.author = author;
            this.copies = copies;
        }
    }

    static class Member {
        String name;

        Member(String name) {
```

```
        this.name = name;
    }
}
```

```
static ArrayList<Book> catalog = new ArrayList<>();
static ArrayList<Member> subscribers = new ArrayList<>();
static Scanner scanner = new Scanner(System.in);

public static void main(String[] args) {

    catalog.add(new Book("The Hobbit", "J.R.R. Tolkien", 4));
    catalog.add(new Book("To Kill a Mockingbird", "Harper Lee", 2));
    catalog.add(new Book("1984", "George Orwell", 3));
    subscribers.add(new Member("Liam Carter"));

    int choice = 0;
    while (choice != 6) {
        System.out.println("\n--- Library Directory System ---");
        System.out.println("1. Show Catalog");
        System.out.println("2. Add New Member");
        System.out.println("3. Show Registered Members");
        System.out.println("4. Issue Book (By Number)");
        System.out.println("5. Return Book (By Number)");
        System.out.println("6. Exit System");
        System.out.print("Enter command: ");
    }
}
```

```

try {
    choice = Integer.parseInt(scanner.nextLine());
} catch (NumberFormatException e) {
    System.out.println("Error: Please input a valid selection
number.");
    continue;
}

if (choice == 1) {
    displayCatalog();
} else if (choice == 2) {
    System.out.print("Enter member's full name: ");
    String name = scanner.nextLine();
    subscribers.add(new Member(name));
    System.out.println("Successfully added member: " + name);
} else if (choice == 3) {
    displayMembers();
} else if (choice == 4) {
    displayCatalog();
    System.out.print("Enter book list number to borrow: ");
    try {
        int num = Integer.parseInt(scanner.nextLine());
        borrowAsset(num - 1);
    } catch (NumberFormatException e) {
        System.out.println("Error: Must enter an integer.");
    }
} else if (choice == 5) {

```

```

        displayCatalog();

        System.out.print("Enter book list number to return: ");

        try {

            int num = Integer.parseInt(scanner.nextLine());

            returnAsset(num - 1);

        } catch (NumberFormatException e) {

            System.out.println("Error: Must enter an integer.");

        }

    } else if (choice == 6) {

        System.out.println("Shutting down directory interface...");

    } else {

        System.out.println("Invalid selection. Try again.");

    }

}

}

```

```

static void displayCatalog() {

    System.out.println("\n=== Current Book Inventory ===");

    if (catalog.isEmpty()) {

        System.out.println("Inventory is currently empty.");

        return;

    }

    for (int i = 0; i < catalog.size(); i++) {

        Book b = catalog.get(i);

        System.out.println((i + 1) + ". " + b.title + " | Author: " +
b.author + " (In Stock: " + b.copies + ")");

    }

}

```

```
    }  
}
```

```
static void displayMembers() {  
    System.out.println("\n=== Registered Members ===");  
    if (subscribers.isEmpty()) {  
        System.out.println("No active members are registered.");  
        return;  
    }  
    for (int i = 0; i < subscribers.size(); i++) {  
        Member m = subscribers.get(i);  
        System.out.println((i + 1) + ". " + m.name);  
    }  
}
```

```
static void borrowAsset(int index) {  
    if (index < 0 || index >= catalog.size()) {  
        System.out.println("Error: That book index does not exist.");  
        return;  
    }  
    Book b = catalog.get(index);  
    if (b.copies > 0) {  
        b.copies = b.copies - 1;  
        System.out.println("Success! You have checked out: " + b.title);  
    } else {
```



```
        System.out.println("Sorry, all copies of \"" + b.title + "\" are  
checked out.");  
    }  
}  
  
static void returnAsset(int index) {  
    if (index < 0 || index >= catalog.size()) {  
        System.out.println("Error: That book index does not exist.");  
        return;  
    }  
  
    Book b = catalog.get(index);  
  
    b.copies = b.copies + 1;  
  
    System.out.println("Success! Returned copy of: " + b.title);  
}  
}
```

2.2 Version B – AI-assisted Programming Approach

```
import java.util.ArrayList;

// Individual class modeling library book entities
class Book {
    String bookId;
    String title;
    String author;
    boolean isAvailable;

    Book(String bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
        this.isAvailable = true;
    }

    void displayDetails() {
        String status = isAvailable ? "In Library" : "Checked Out";
        System.out.println "[" + bookId + "] " + title + " by " + author + "
Status: " + status);
    }
}

// Individual class modeling reader profiles
class Member {
```

```

String memberId;

String name;

ArrayList<Book> activeLoans = new ArrayList<>();

Member(String memberId, String name) {
    this.memberId = memberId;
    this.name = name;
}

void borrowBook(Book book) {
    if (!book.isAvailable) {
        System.out.println(name + " cannot borrow \"" + book.title + "\"
(Currently unavailable).");
        return;
    }
    book.isAvailable = false;
    activeLoans.add(book);
    System.out.println(name + " has successfully borrowed \"" +
book.title + "\".");
}

void returnBook(Book book) {
    if (!activeLoans.contains(book)) {
        System.out.println("Record Error: " + name + " did not borrow \""
+ book.title + "\".");
        return;
    }
}

```

```

        book.isAvailable = true;

        activeLoans.remove(book);

        System.out.println(name + " has successfully returned \"" +
book.title + "\".");
    }

    void viewIssuedItems() {
        System.out.println("Books checked out by " + name + ":");
        if (activeLoans.isEmpty()) {
            System.out.println(" None.");
        } else {
            for (Book b : activeLoans) {
                System.out.println(" * " + b.title);
            }
        }
    }
}

```

// Coordinating controller class

```

class Library {
    ArrayList<Book> bookList = new ArrayList<>();
    ArrayList<Member> memberList = new ArrayList<>();

    void registerBook(Book book) {
        bookList.add(book);

        System.out.println("Log: Book registered (" + book.title + ")");
    }
}

```

```
void registerMember(Member member) {  
    memberList.add(member);  
    System.out.println("Log: Member registered (" + member.name + ")");  
}
```

```
Book findBook(String id) {  
    for (Book b : bookList) {  
        if (b.bookId.equalsIgnoreCase(id)) {  
            return b;  
        }  
    }  
    return null;  
}
```

```
Member findMember(String id) {  
    for (Member m : memberList) {  
        if (m.memberId.equalsIgnoreCase(id)) {  
            return m;  
        }  
    }  
    return null;  
}
```

```
void printFullCatalog() {  
    System.out.println("\n--- Current Library Asset Catalog ---");  
    if (bookList.isEmpty()) {
```

```

        System.out.println("No records in directory.");
        return;
    }
    for (Book b : bookList) {
        b.displayDetails();
    }
}

}

// Primary execution demo class
public class LibrarySystem {
    public static void main(String[] args) {
        Library library = new Library();

        // 1. Log in active assets
        System.out.println("=== Step 1: Initializing Assets ===");
        Book b1 = new Book("B101", "The Catcher in the Rye", "J.D.
Salinger");
        Book b2 = new Book("B102", "The Great Gatsby", "F. Scott
Fitzgerald");
        Book b3 = new Book("B103", "Fahrenheit 451", "Ray Bradbury");

        library.registerBook(b1);
        library.registerBook(b2);
        library.registerBook(b3);

        System.out.println("\n=== Step 2: Registering Users ===");
    }
}

```

```
Member m1 = new Member("M201", "Sophia Bennett");
Member m2 = new Member("M202", "Ethan Hunt");

library.registerMember(m1);
library.registerMember(m2);

// View baseline database
library.printFullCatalog();

System.out.println("\n=== Step 3: Simulating Borrow Operations ===");

m1.borrowBook(b1);
m2.borrowBook(b1);
m2.borrowBook(b2);

library.printFullCatalog();

System.out.println();
m1.viewIssuedItems();
m2.viewIssuedItems();

System.out.println("\n=== Step 4: Simulating Return Operations ===");
m1.returnBook(b1);

library.printFullCatalog();
}
}
```

Prompt: Design and implement a Library Management System in Java following object-oriented design principles. The system must include a Book class to represent book attributes and availability, a Member class to represent member details and borrowing behaviour, and a Library class to coordinate book and member management.

2.3 Comparison of Version A and Version B

Criteria	Version A	Version B
Development time	≈ 3 hours	≈ 45 minutes initial draft + 30 minutes review/fixes
Code quality	Functional but minimal validation (basic email check)	More robust: regex validation, type hints, dataclasses
Error handling	Basic ValueError / KeyError checks	Similar exceptions, plus clearer messages generated by AI
Readability and maintainability	Readable but verbose; no type hints	More concise and self-documenting due to type hints and docstrings

2.4 UML Diagram

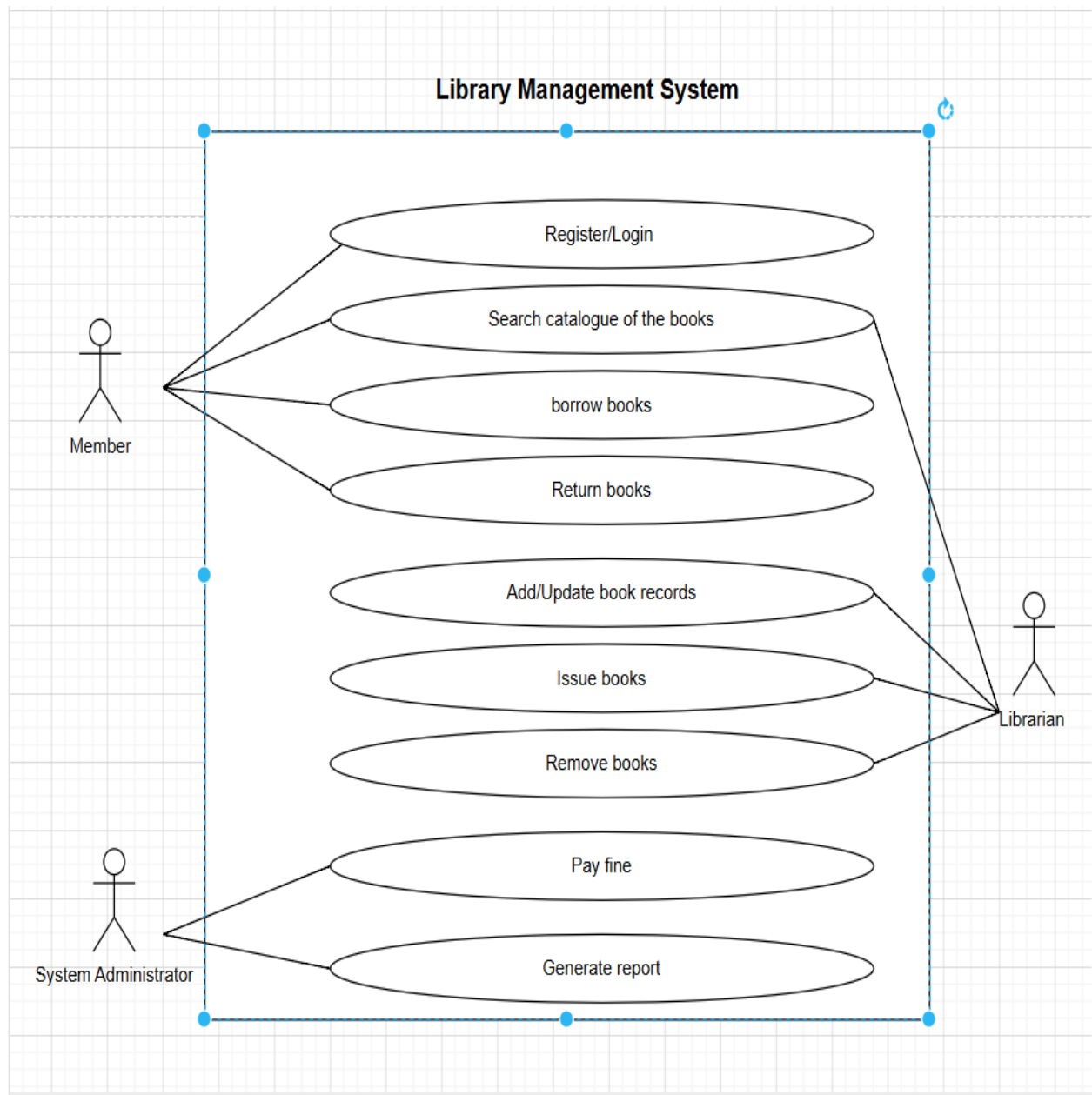


Figure 1: Use Case diagram

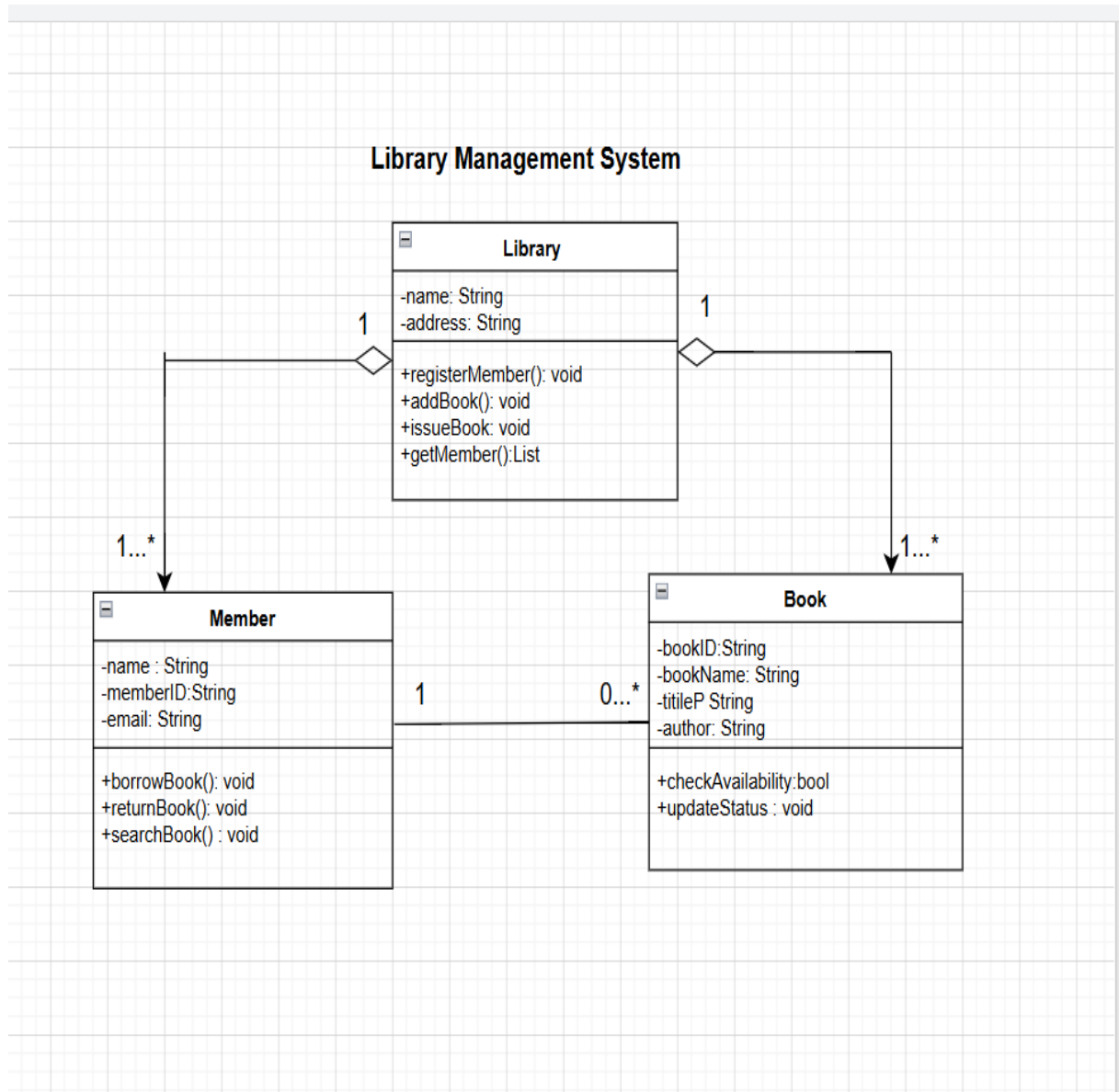


Figure 2 : Class diagram

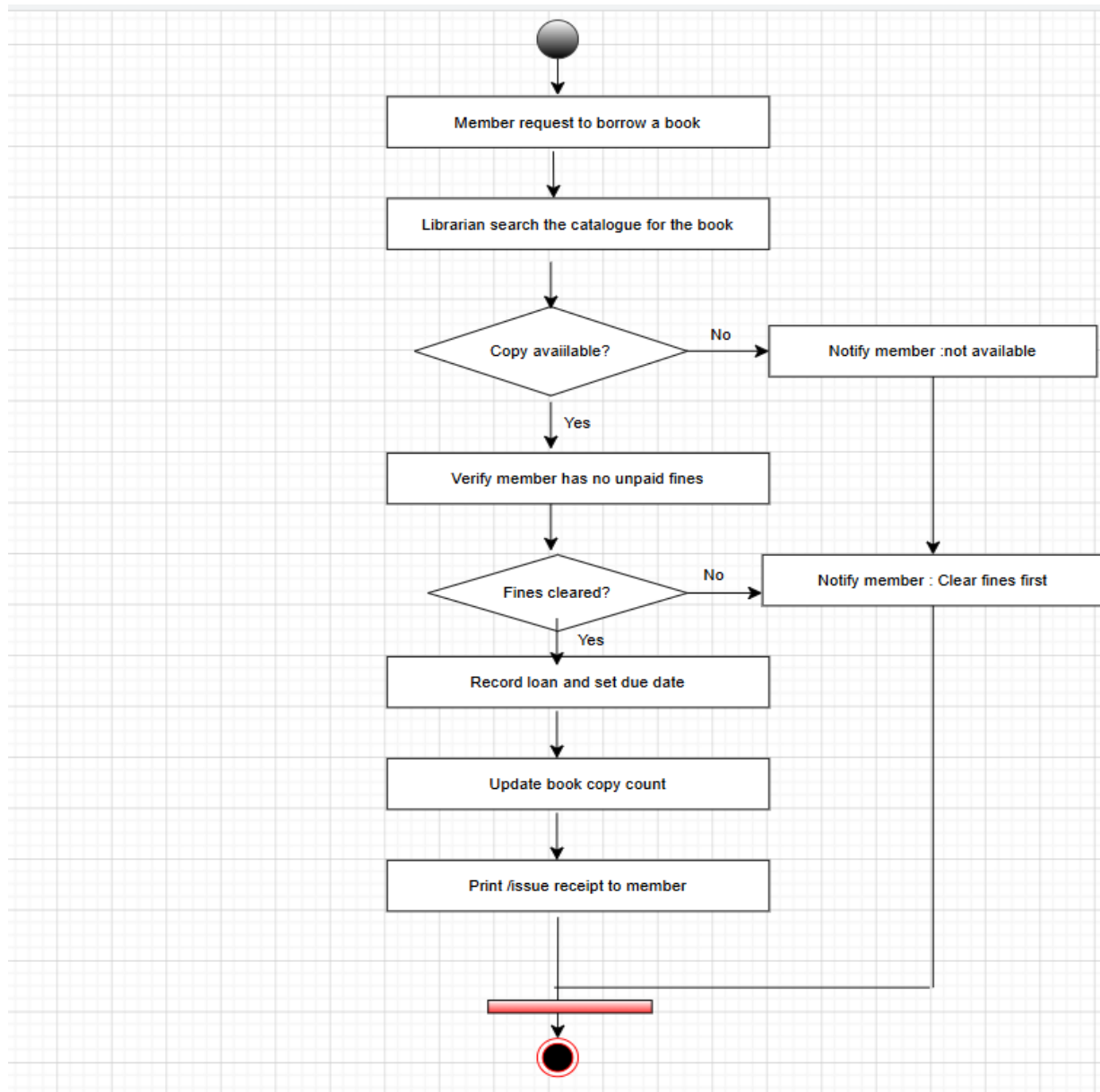


Figure 3 : Activity diagram

Task 3: Critical Analysis

3.1 How AI Changed the Coding Approach

Working with an AI assistant shifted the development process from writing every line manually to a more iterative, conversational style. Instead of consulting documentation line-by-line, the workflow became: describe the requirement in natural language, review the AI's suggestion, test it, and refine it. This encouraged thinking more about what the code should do (specification) and less about how to type each syntactic detail, especially for repetitive patterns like validation logic and data classes.

3.2 Did AI Improve or Reduce Understanding?

AI assistance had a mixed effect on understanding. On one hand, it introduced useful patterns — such as using dataclasses and regular expressions — that the student had not used confidently before, and explaining these on request improved conceptual understanding. On the other hand, it was tempting to accept a working suggestion without fully tracing through its logic, which is how the bug in `update_student` went unnoticed until it was specifically reviewed. This suggests that AI tools can both accelerate learning of new patterns and, if used passively, reduce the depth of understanding of the resulting code.

3.3 When AI Should NOT Be Used

- When working with security-critical code (authentication, payment processing, access control), where subtle AI-introduced flaws could have serious consequences.
- When learning a concept for the first time and the goal is to build genuine understanding rather than working code — writing it manually first reinforces learning.
- In safety-critical or highly regulated systems (medical devices, aviation, financial systems) where code must be traceable, certified, and fully understood by the development team.

- When the AI's training data may not reflect the specific, up-to-date version of a library or framework, risking the use of deprecated or non-existent APIs.
- When intellectual-property or licensing constraints require complete certainty about the origin of every line of code.

3.4 Ethical Considerations in AI-Generated Code

- Attribution and transparency — developers should disclose which parts of a codebase were AI-generated, particularly in academic or collaborative settings, to maintain honesty about authorship.
- Accountability — a human developer remains responsible for the correctness and safety of code they ship, even if it was generated by an AI tool; "the AI wrote it" is not a valid excuse for a shipped bug.
- Bias and fairness — AI models are trained on large public codebases and may reproduce insecure or biased patterns present in that training data.
- Intellectual property — there is ongoing debate about whether AI-generated code may inadvertently reproduce copyrighted material from training data, which raises licensing risk for commercial projects.
- Academic integrity — in an educational context, students must be careful to use AI as a learning aid rather than a substitute for demonstrating their own understanding, in line with their institution's academic integrity policy.

Task 4: Conclusion

This assignment demonstrated that AI-assisted programming is a powerful productivity tool rather than a replacement for a developer's own understanding and judgement. Building the Student Management System twice — once manually and once with AI assistance — made the trade-offs concrete: the AI-assisted version was completed in roughly a third of the time and introduced cleaner patterns such as type hints and structured validation, but it also introduced a logic bug that would have gone unnoticed without careful manual review.

In an educational context, AI tools are most valuable when used to accelerate routine work and to explain unfamiliar concepts, while core logic, testing, and final review are still performed by the student, so that the skill of reasoning about code is not lost. In an industry context, the same principle applies at a larger scale: AI-assisted programming can significantly increase throughput, but organisations should pair it with strong code review practices, security scanning, and clear policies on where AI-generated code is and is not appropriate, particularly for critical systems. Overall, AI-assisted programming should be viewed as a collaborative tool that augments a developer's capabilities rather than a substitute for programming knowledge, critical thinking, and professional responsibility.

GitHub Repository : <https://github.com/dilkinadeeshara22/Library-Management-System>